串口全彩灯带控制器

手 册

通信协议

天津创联容微科技有限公司

专业的电子技术软硬件开发制造商 . . .*make electronics easy to use and learn.*

目录

一、产品简介 2
二、指令格式 2
2.1、组地址 2
2.2、设备地址 3
2.3、组地址和设备地址的结合 4
2.4、端口号 5
2.5、功能码 7
2.5.1、查询控制器信息反馈设置状态（8B）（V2.0 固件版本以上才支持） 8
2.5.2、修改控制器信息反馈设置状态（8C）（V2.0 固件版本以上才支持） 9
2.5.3、查询控制器串口通信超时时间（8D） 10
2.5.4、修改控制器串口通信超时时间（8E） 11
2.5.5、查询控制器唯一序列号（8F） 12
2.5.6、通过控制器唯一序列号查询控制器设备地址（90） 12
2.5.7、通过控制器唯一序列号修改控制器设备地址（91） 13
2.5.8、查询总线上有没有与指令中组地址和设备地址匹配的控制器（92） 14
2.5.9、查询控制器组地址（93） 14
2.5.10、查询控制器设备地址（94） 15
2.5.11、修改控制器通信波特率（95） 16
2.5.12、修改控制器组地址（96） 17
2.5.13、修改控制器设备地址（97） 17
2.5.14、显示颜色数据（99） 18
2.5.15、设置上电显示的颜色（9B） 21
2.5.16、关闭上电显示功能（9C） 22
2.5.17、按 IC 索引修改显示数据（98）（V2.0 固件版本以上才支持） 22
2.5.18、读取上电显示的数据（9D） 24
2.6、灯带类型 26
2.7、数据长度 28
2.8、扩展次数 29
2.9、颜色数据 29
2.10、当指令错误时控制器的回复内容 30
三、颜色显示指令范例 31
四、如何在程序中如何构造一条显示指令 32
五、其他说明 36
六、联系我们 37

一、产品简介

这款串口全彩灯带控制器，可以实现主机发送指令和颜色数据给控制器，控制器把颜色数据显示在灯带上。主机可以是 PC，单片机，或者其他的带串口的设备。控制器默认组地址 0，默认设备地址 1，默认波特率 115200

二、指令格式

名称	指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分	
字节数	3			2		2		1	1	1	2	
索引	0	1	2	3	4	5	6	7	8	9	10	11
示例	DD	55	EE	00	00	00	01	00	99	01	00	00

数据长度		扩展次数		颜色数据			指令尾	
2		2		3~3072			2	
12	13	14	15	16	17	18	19	20
00	03	00	01	FF	00	00	AA	BB

备注：索引一行中的数字是十进制数，示例一行中的数字每一个是一个 16 进制数。

指令由 11 个部分组成，其中黄色标注的部分是固定的内容，不会改变。下面分别对每一部分进行说明。

2.1、组地址

名称	组地址	
字节数	2	
索引	3	4
示例	00	00

组地址占 2 个字节，位于指令中的第 3，第 4 字节。

组地址取值范围：0~1024，这是一个十进制数，使用时要用两个字节表示，高有效字节在前。比如地址是 1，则分解成 00　01。如果地址是 5，则分解成 00　05，如果地址是 256，则分解成 01　00。

组地址的作用：

当使用 485 接口的控制器时，在一组 485 总线上可以连接多个设备，给每个设备分配一个独立的设备地址。主机发送指令时，只有设备地址匹配的设备才会响应指令。这样可以控制每个设备所连接的灯带显示不同的颜色或效果。

假设此时总线上连接了 10 个设备，每个设备有不同的地址。当想让其中的 00 02 号和 00 04 号设备显示相同的效果时，需要发两条指令，给 00 02 号发一条，然后给 00 04 号发一条，这时导致两个设备不能同步显示。

为了解决这个问题，在指令中添加了组地址。每个设备除了有一个设备地址外，还有一个组地址。解决上面问题的方法是给 00 02 号设备和 00 04 号设备分配一个相同的组地址，比如是 00 01 组。这时就可以针对这一组设备发指令，485 总线中的设备只要组地址是 00 01，都会同时响应这条指令。

组地址的作用范围：当组地址是 00 01 的时候，表示当前指令只对组地址是 00 01 的设备有效。当组地址是 00 05 的时候，表示当前指令只对组地址是 00 05 的设备有效。类推。

特殊的组地址取值：当组地址是 00 00 的时候，表示当前指令对当前总线上连接的所有设备有效。

2.2、设备地址

名称	设备地址	
字节数	2	
索引	5	6
示例	00	01

设备地址占 2 个字节，位于指令中的第 5，第 6 字节。

设备地址取值范围：0~4096，这是一个十进制数，使用时要用两个字节表示，高有效字节在前。比如地址是 1，则分解成 00 01。如果地址是 5，则分解成 00 05，如果地址是 256，则分解成 01 00。

设备地址的作用：

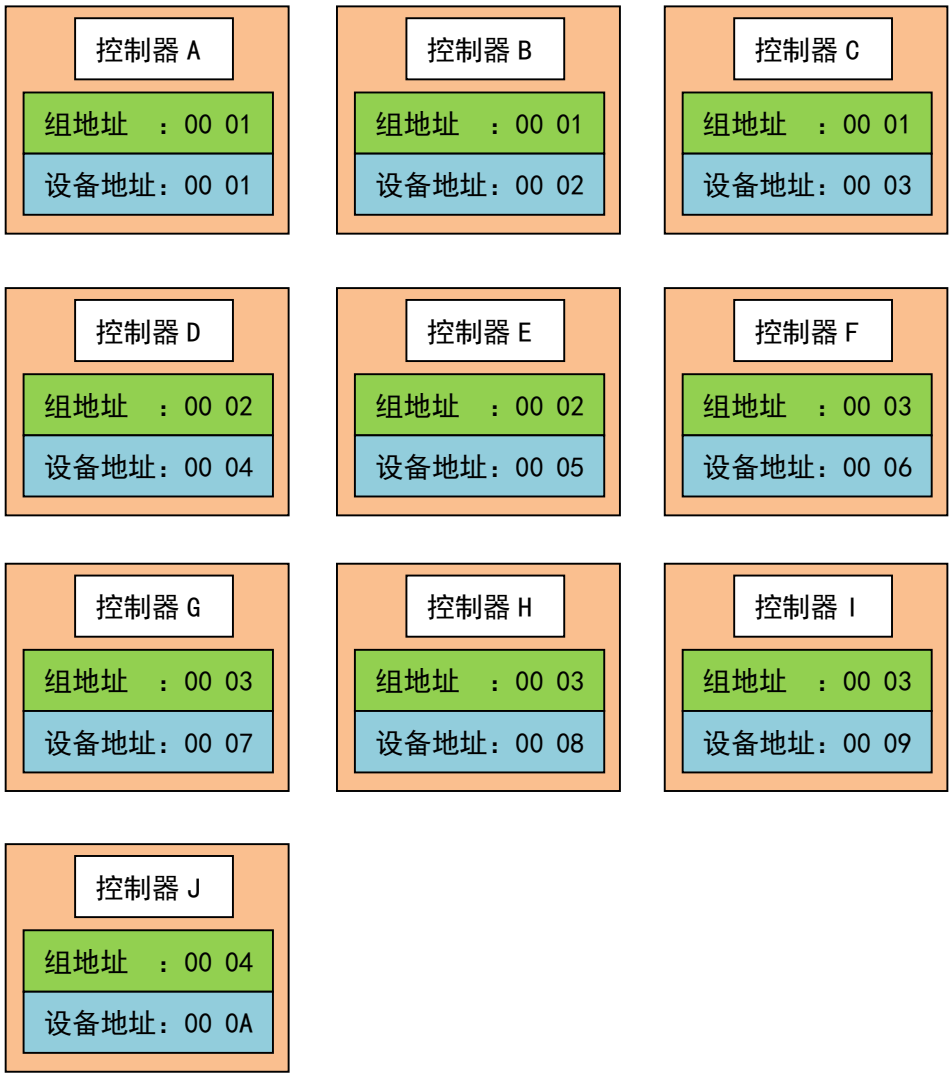
当使用 485 接口的控制器时，在一组 485 总线上可以连接多个设备，为了区分每个设备，所以给每个设备分配一个设备地址，任意两个设备的设备地址不能相同。

设备地址的作用范围：当设备地址是 00 01 的时候，表示当前指令只对设备地址是 00 01 的设备有效。当设备地址是 00 03 的时候，表示当前指令只对设备地址是 00 03 的设备有效。类推。

特殊的设备地址取值：当设备地址是 00 00 的时候，表示当前指令对当前总线上连接的所有设备有效。

2.3、组地址和设备地址的结合

假设在一条总线上连接了下面 10 个控制器：



第一种情况：当指令的组地址是 00 01，设备地址是 00 01 的时候，此条指令只对控制器 A 有效。指令的组地址是 00 01，设备地址是 00 02 的时候，此条指令只对控制器 B 有效。指令的组地址是 00 03，设备地址是 00 07 的时候，此条指令只对控制器 G 有效。类推。

（结论：指令的组地址不是 00 00 ，设备地址也不是 00 00，此指令只对组地址和设备地址同时相符的控制器有效）

第二种情况：当指令的组地址是 00 02，设备地址是 00 02 的时候，此条指令对任何控制器都无效，因为没有相符的设备。当指令的组地址是 00 01，设备地址是 00 08 的时候，此条指令对任何控制器都无效，因为没有相符的设备。类推。

第三种情况：当指令的组地址是 00 01，设备地址是 00 00 的时候，此条指令对控制器 A、控制

器 B、控制器 C 有效。当指令的组地址是 00 02，设备地址是 00 00 的时候，此条指令对控制器 D、控制器 E、控制器 F 有效。类推。

(结论：指令的组地址不是 00 00，设备地址是 00 00，则此指令对组地址相符的控制器都有效)。

第四种情况：当指令的组地址是 00 00，设备地址是 00 01 的时候，此条指令只对控制器 A 有效。当指令的组地址是 00 00，设备地址是 00 08 的时候，此条指令只对控制器 H 有效。类推。(结论：指令的组地址是 00 00，设备地址不是 00 00，则此指令对设备地址相符的控制器有效。)

第五种情况：当指令的组地址是 00 00，设备地址是 00 00 的时候，对控制器 A、B、C、D、E、F、G、H、I、J 都有效。

2.4、端口号

名称	端口号
字节数	1
索引	7
示例	00

端口号占 1 个字节，位于指令中的第 7 字节。

设备地址取值范围：0~6，这是一个十进制数，使用时要用转换成 16 进制数。

端口号的作用：每个控制器可以有多个端口，不同的端口可以连接不同的灯带，指令可以指定当前指令在控制器的哪个端口显示。

端口的构成：控制器上一个 DATx 引脚和一个 CLKx 引脚表示一个端口。(或者标注 DO，表示一个端口)

端口号各个值的作用范围（绿色填充表示有效范围）：

	控制器端口					
取值	1 号	2 号	3 号	4 号	5 号	6 号
00	■	■	■	■	■	■
01	■					
02		■				
03			■			
04				■		
05					■	
06						■

取值：对于只有一个端口的控制器，取值可以是 00，或者 01，当取其他值的时候无效，指令不能正常显示。对于只有两个端口的控制器，取值可以是 00，或者 01，或者 02，当取其他值的时候无效，指令不能正常显示。

2.5、功能码

名称	功能码
字节数	1
索引	8
示例	99

功能码占 1 个字节，位于指令中的第 8 字节。

功能码取值范围：8B~9D，这是一个 16 进制数。不同的取值代表指令不同的功能，比如取值是 99 的时候是显示指令。取值 97 的时候是修改设备地址指令。具体如下：

取值	功能
8B	查询控制器的信息反馈设置状态（V2.0 以上才支持）
8C	修改控制器的信息反馈设置状态（V2.0 以上才支持）
8D	查询控制器串口通信超时时间（V1.7 以上才支持）
8E	修改控制器串口通信超时时间（V1.7 以上才支持）
8F	查询控制器唯一序列号
90	通过控制器唯一序列号查询控制器设备地址
91	通过控制器唯一序列号修改控制器设备地址
92	查询总线上有没有与指令中组地址和设备地址匹配的控制器
93	查询控制器组地址
94	查询控制器设备地址
95	修改控制器通信波特率
96	修改控制器组地址
97	修改控制器设备地址
98	按 IC 索引修改显示数据（V2.0 以上才支持）
99	显示颜色数据
9A	等同于显示颜色数据
9B	设置上电显示的数据
9C	关闭上电显示功能
9D	读取上电显示的数据（V1.9 以上才支持）

注意：由于所有指令使用相同的数据缓冲区接收指令。使用 98 指令的过程中（比如使用 98 指令做实现一些动态效果），如果穿插使用了其他指令，会破坏数据缓冲区中显示的数据。导致显示数据被修改。

下面对每一个功能码分别介绍：

2.5.1、查询控制器信息反馈设置状态（8B）（V2.0 固件版本以上才支持）

当主机的串口总线同时连接了灯带控制器和其他串口设备时，如果控制器收到指令后每次都回复信息，可能会干扰串口总线上其他设备的通信，或者影响通信效率。在确定主机和灯带控制器通信完全可靠，已经调试好后，可以关闭控制器收到指令后的回复信息。

信息回复设置分 3 级：

F0 级，表示控制器收到指令后会正常回复信息。

F1 级，表示控制器收到指令后，不会回复“recvEnd\r\n”，”DisplayEnd\r\n”，”saveInsEnd\r\n”.

F2 级，除了不回复 F1 列出的信息外，即使控制器收到格式错误的指令也不回复错误码（xERR），如果指令不完整通信超时，也不回复“timeout\r\n”.

控制器默认通信回复信息设置是 F0，这个值一般不需要修改。将回复信息设置修改为 F1 或者 F2 不利于控制器调试。

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	01	00	8B	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，设备地址填写想要查询的控制器的设备地址。

示例 1： DD 55 EE 00 00 00 01 00 8B 01 00 00 00 03 00 01 00 00 00 AA BB

这条指令表示查询设备地址是 00　01 的控制器的串口回复信息设置。

示例 2： DD 55 EE 00 00 00 0000 8B 01 00 00 00 03 00 01 00 00 00 AA BB

当总线上只连接了一个控制器时，可以使用上面这条指令查询控制器的串口回复信息设置。

示例 3： DD 55 EE 00 01 00 0100 8B 01 00 00 00 03 00 01 00 00 00 AA BB

上面这条指令的是查询组地址是 00　01，设备地址是 00　01 的控制器的串口回复信息设置，控制器默认通信回复信息设置是 F0，这个值一般不需要修改。将回复信息设置修改为 F1 或者 F2 不利于控制器调试。

指令的回复：

当有匹配的设备时会收到回复，以字符形式，示例如下：

F0\r\n

表示所查询的设备的通信回复信息设置是 F0。

2.5.2、修改控制器信息反馈设置状态（8C）（V2.0 固件版本以上才支持）

当主机的串口总线同时连接了灯带控制器和其他串口设备时，如果控制器收到指令后每次都回复信息，可能会干扰串口总线上其他设备的通信，或者影响通信效率。在确定主机和灯带控制器通信完全可靠，已经调试好后，可以关闭控制器收到指令后的回复信息。

信息回复设置分 3 级：

F0 级，表示控制器收到指令后会正常回复信息。

F1 级，表示控制器收到指令后，不会回复“recvEnd\r\n”，“DisplayEnd\r\n”，“saveInsEnd\r\n”.

F2 级，除了不回复 F1 列出的信息外，即使控制器收到格式错误的指令也不回复错误码（xERR），如果指令不完整通信超时，也不回复“timeout\r\n”.

控制器默认通信回复信息设置是 F0，这个值一般不需要修改。将回复信息设置修改为 F1 或者 F2 不利于控制器调试。

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	01	00	8C	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，设备地址填写想要设置信息回复状态的控制器的设备地址。

示例 1： DD 55 EE 00 00 00 01 00 8C 01 00 00 00 03 00 01 00 00 F0AA BB

这条指令表示将设备地址是 00　01 的控制器的串口回复信息设置为 F0。

示例 2： DD 55 EE 00 00 00 00 00 8D 01 00 00 00 03 00 01 00 00 F0AA BB

当总线上只连接了一个控制器时，可以使用上面这条指令设置控制器的串口回复信息设置为 F0。

示例 3： DD 55 EE 00 01 00 0100 8D 01 00 00 00 03 00 01 00 00 F1AA BB

上面这条指令的是设置组地址是 00　01，设备地址是 00　01 的控制器的串口回复信息设置为 F1，

控制器默认通信回复信息设置是 F0，这个值一般不需要修改。将回复信息设置修改为 F1 或者 F2 不利于控制器调试。

指令的回复：

当有匹配的设备时会收到回复，以字符形式，示例如下：

F0\r\n

表示所设置的设备通信回复信息设置已修改为 F0。

2.5.3、查询控制器串口通信超时时间（8D）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	01	00	8D	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，设备地址填写想要查询的控制器的设备地址。

示例 1： DD 55 EE 00 00 00 01 00 8D 01 00 00 00 03 00 01 00 00 00 AA BB

这条指令表示查询设备地址是 00　01 的控制器的串口通信超时时间。

示例 2： DD 55 EE 00 00 00 0000 8D 01 00 00 00 03 00 01 00 00 00 AA BB

当总线上只连接了一个控制器时，可以使用上面这条指令查询控制器的串口通信超时时间。

示例 3： DD 55 EE 00 01 00 0100 8D 01 00 00 00 03 00 01 00 00 00 AA BB

上面这条指令的是查询组地址是 00　01，设备地址是 00　01 的控制器的串口通信超时时间，控制器默认通信超时时间是 5ms，这个值一般不需要修改。一般情况下这个值不是越大越好。

指令的回复：

当有匹配的设备时会收到回复，以字符形式，示例如下：

0005\r\n

表示所查询的设备的通信超时时间是 5ms。

2.5.4、修改控制器串口通信超时时间（8E）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		新波特率			指令尾	
DD	55	EE	00	00	00	00	00	8E	01	00	00	00	03	00	01	00	00	05	AA	BB

黄色和蓝色的部分的内容都是固定的，组地址和设备地址填写想要修改的控制器的组地址和设备地址。由于一个总线上控制器的通信超时时间应该是一致的，所以指令中的组地址和设备地址可以都是 00　00。如果指令的组地址和设备地址不是 00　00，则匹配规格遵循《**2.3、组地址和设备地址的结合**》。

控制器串口通信超时时间默认值是 5ms，可修改的范围是 5ms 到 1000ms。大部分情况下，主机向控制器发送的每条指令都是一次性发送的，中间不会断开，所以不需要修改控制器的通信超时时间。

只有在受主机硬件限制，主机做不到把一条显示指令一次性发送完时，也就是一条指令被分成多段发送时。这时如果收到的控制器反馈是”timeout”,那么就需要修改控制器的通信超时时间。

举例：使用网口转串口设备给控制发送命令时，单条 IP 包最长 1492 个字节，如果显示指令长度超过了 1492 字节，就会被网络分成多次发送，此时可能收到控制器回复 timeout，这时可以尝试修改控制器通信超时时间。

示例指令如下：

示例 1： DD 55 EE 00 00 00 00 00 8E 01 00 00 00 03 00 01 000020 AA BB

把所有控制器通信超时时间修改为 0x20（也就是 32ms）

示例 3： DD 55 EE 00 00 00 00 00 8E 01 00 00 00 03 00 010000FF AA BB

把所有控制器通信超时时间修改为 0x00FF（也就是 255ms）

示例 2： DD 55 EE 00 00 00 00 00 8E 01 00 00 00 03 00 01 00 0400 AA BB

把所有控制器通信超时时间修改为 0x0400（也就是 1000ms）

提示：修改通信超时时间后立即生效。

指令的回复：

如果指令中的组地址不是 00　00，设备地址也不是 00　00 ，如果指令正确，控制器接收完指令后会回复：

RecvEnd\r\n

如果超时时间修改成功，控制器会回复修改后的超时时间，比如回复 50\r\n，就是控制器的通信超时时间已经被修改为 50ms。

2.5.5、查询控制器唯一序列号（8F）

每个控制器都有一个 96bit（12 个字节）的唯一序列号，可以通过查询获得。

使用条件：总线上只连接一个控制器或总线上只有一个设备与指令的组地址和设备地址匹配。

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	00	00	8F	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，组地址和设备地址部分参考《**2.3、组地址和设备地址的结合**》

去掉表格后：

DD 55 EE 00 00 00 00 00 8F 01 00 00 00 03 00 01 00 00 00 AA BB

发送指令后收到的返回值：

当总线中只连接了一个设备，或总线中只有一个控制器与指令中指定的组地址和设备地址匹配时，会收到 14 个字节数据回复，其中前 12 个字节是控制器唯一序列号，第 13 个字节是 0D（\r），第 14 个字节是 0A（\n）。

示例：36 00 2A 00 13 53 4D 52 39 34 37 20 0D 0A，表示设备唯一序列号是：36 00 2A 00 13 53 4D 52 39 34 37 20

2.5.6、通过控制器唯一序列号查询控制器设备地址（90）

使用条件：必须在已知总线上控制器序列号的情况下使用

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数	
DD	55	EE	00	00	00	00	00	90	01	00	00	00	0C	00	01

序列号数据												指令尾	
00	00	00	00	00	00	00	00	00	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，白色的部分根据已知的序列号填写。

示例：DD 55 EE 00 00 00 00 00 90 01 00 00 00 0C 00 01 36 00 2A 00 13 53 4D 52 39 34 37 20 AA BB

指令的回复：

当总线上有与序列号匹配的控制器时可以收到回复，以字符形式回复，示例如下：

0001\r\n

其中 0001 是控制器的设备地址，表示控制器设备地址是 1，\r\n 是回车换行符。假设控制器设备地址是 23，则收到的是：0023\r\n

2.5.7、通过控制器唯一序列号修改控制器设备地址（91）

使用条件：必须在已知总线上控制器序列号的情况下使用

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数	
DD	55	EE	00	00	00	00	00	91	01	00	00	00	0F	00	01

序列号数据												新设备地址			指令尾	
00	00	00	00	00	00	00	00	00	00	00	00	00	00	02	AA	BB

黄色和蓝色的部分的内容都是固定的，白色的部分根据已知的序列号填写，绿色的部分根据需要设置的设备地址值进行设置，高有效字节在前。

示例：DD 55 EE 00 00 00 00 00 91 01 00 00 00 0F 00 01 36 00 2A 00 13 53 4D 52 39 34 37 20 00 00 02 AA BB

指令的回复：

当总线上有与序列号匹配的控制器时可以收到回复，以字符形式回复，示例如下：

0002\r\n

其中 0002 是修改后的控制器的设备地址，表示控制器设备地址是 2，\r\n 是回车换行符。

2.5.8、查询总线上有没有与指令中组地址和设备地址匹配的控制器（92）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	01	00	92	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，组地址和设备地址填写想要查询的控制器的组地址和设备地址。

示例 1： DD 55 EE 00 00 00 01 00 92 01 00 00 00 03 00 01 00 00 00 AA BB

上面指令表示查询有没有设备地址是 00　01 的控制器存在。因为指令中组地址是 00　00，所以组地址被忽略，只要设备地址匹配就会收到回复。

示例 2： DD 55 EE 00 01 00 01 00 92 01 00 00 00 03 00 01 00 00 00 AA BB

上面指令表示查询有没有组地址是 00　01，同时设备地址是 00　01 的控制器存在。只有总线上有组地址和设备地址同时匹配的控制器存在时才会收到回复。

指令的回复：

有匹配的设备时，以字符形式回复：

YES\r\n

\r\n 是回车换行符。只需要在接收到的字符串中检查有没有“YES”

2.5.9、查询控制器组地址（93）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	01	00	93	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，设备地址填写想要查询的控制器的设备地址。

示例 1： DD 55 EE 00 00 00 01 00 93 01 00 00 00 03 00 01 00 00 00 AA BB

这条指令表示查询设备地址是 00　01 的控制器的组地址。

示例 2： DD 55 EE 00 00 00 0000 93 01 00 00 00 03 00 01 00 00 00 AA BB

当总线上只连接了一个控制器时，可以使用上面这条指令查询控制器的组地址。

示例 3： DD 55 EE 00 01 00 0100 93 01 00 00 00 03 00 01 00 00 00 AA BB

上面这条指令的是查询组地址是 00　01，设备地址是 00　01 的控制器的组地址，由于指令中指定了组地址，所以实际上这条指令的作用相当于**《5.4、查询总线上有没有与指令中组地址和设备地址匹配的控制器（92）》**

指令的回复：

当有匹配的设备时会收到回复，以字符形式，示例如下：

0001\r\n

表示所查询的设备的组地址是 1。

2.5.10、查询控制器设备地址（94）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	00	00	94	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，设备地址填写想要查询的控制器的设备地址。

示例 1： DD 55 EE 00 00 00 0000 94 01 00 00 00 03 00 01 00 00 00 AA BB

当总线上只连接了一个控制器时，可以使用上面这条指令查询控制器的设备地址。

示例 2： DD 55 EE 00 01 00 0000 94 01 00 00 00 03 00 01 00 00 00 AA BB

这条指令表示查询组地址是 00　01 的控制器的设备地址。

示例 3： DD 55 EE 00 01 00 0100 94 01 00 00 00 03 00 01 00 00 00 AA BB

上面这条指令的是查询组地址是 00　01，设备地址是 00　01 的控制器的设备地址，由于指令中指定了设备地址，所以实际上这条指令的作用相当于**《5.4、查询总线上有没有与指令中组地址和设备地址匹配的控制器（92）》**

指令的回复：

当有匹配的设备时会收到回复，以字符形式，比如收到如下回复：

0002\r\n

表示所查询的设备的设备地址是 2。

2.5.11、修改控制器通信波特率（95）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		新波特率			指令尾	
DD	55	EE	00	00	00	00	00	95	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，组地址和设备地址填写想要修改的控制器的组地址和设备地址。由于一个总线上控制器的通信波特率应该都是和主机一致的，所以指令中的组地址和设备地址应该都是 00 00。如果指令的组地址和设备地址不是 00 00，则匹配规格遵循《**2.3、组地址和设备地址的结合**》。

控制器只支持 9600，115200，230400，460800，921600 五种波特率，默认波特率 115200，其他波特率不支持。所以有效的指令如下：

示例 1： DD 55 EE 00 00 00 00 00 95 01 00 00 00 03 00 01 01 C2 00 AA BB

把所有控制器波特率修改为 115200

示例 2： DD 55 EE 00 00 00 00 00 95 01 00 00 00 03 00 01 00 25 80 AA BB

把所有控制器波特率修改为 9600

示例 3： DD 55 EE 00 00 00 00 00 95 01 00 00 00 03 00 0103 84 00 AA BB

把所有控制器波特率修改为 230400

示例 4： DD 55 EE 00 00 00 00 00 95 01 00 00 00 03 00 0107 08 00 AA BB

把所有控制器波特率修改为 460800

示例 5： DD 55 EE 00 00 00 00 00 95 01 00 00 00 03 00 010E 10 00 AA BB

把所有控制器波特率修改为 921600

提示 1： 修改波特率后，控制器重新上电后生效。

提示 2： 如果修改波特率后，下次再使用时忘记了控制器的波特率，那么只能把这 5 种波特率都实验一次，用哪个波特率能通信上，那么控制器就是哪个波特率。另外控制器每次启动时都会发出一串字符“0000_0001_0005_powerON\r\n”，用哪个波特率能正常收到这串字符，就代表控制器是哪个波特率。

2.5.12、修改控制器组地址（96）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		新组地址			指令尾	
DD	55	EE	00	00	00	01	00	96	01	00	00	00	03	00	01	00	00	02	AA	BB

黄色和蓝色的部分的内容都是固定的，指令的组地址和设备地址部分填写想要修改的控制器的组地址和设备地址。红色部分填写新的组地址。

指令作用：用于给控制器分组。

示例 1：DD 55 EE 00 00 00 01 00 96 01 00 00 00 03 00 01 00 00 02 AA BB

上面指令是把设备地址是 00　01 的控制器的组地址修改为 00　02

示例 2：DD 55 EE 00 00 00 0000 96 01 00 00 00 03 00 01 00 00 01 AA BB

上面指令是把总线上连接的所有控制器的组地址都修改为 00　01

示例 3：DD 55 EE 00 01 00 0000 96 01 00 00 00 03 00 01 00 00 03 AA BB

上面指令是把总线上原来组地址是 00　01 的控制器的组地址修改为 00　03

示例 4：DD 55 EE 00 02 00 0300 96 01 00 00 00 03 00 01 00 00 01 AA BB

上面指令是把总线上原来组地址是 00　02，设备地址是 00　03 的控制器的组地址修改为 00　01

指令的回复：

像上面的示例 2 和示例 3，修改完成后收不到回复，因为是针对所有控制器或者一组控制器进行修改。上面的示例 1 和示例 4，如果有匹配的控制器，修改完后后会收到回复，以字符串形式，示例如下：

0002\r\n

表示组地址修改完成，新的组地址是 2。

提示：组地址修改完成后是立即生效。

2.5.13、修改控制器设备地址（97）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		新设备地址			指令尾	
DD	55	EE	00	00	00	00	00	97	01	00	00	00	03	00	01	00	00	02	AA	BB

黄色和蓝色的部分的内容都是固定的，指令的组地址和设备地址部分填写想要修改的控制器的组地址

和设备地址。红色部分填写新的设备地址。

示例 1：DD 55 EE 00 00 00 01 00 97 01 00 00 00 03 00 01 00 00 02 AA BB

上面指令是把设备地址是 00　01 的控制器的设备地址修改为 00　02

示例 2：DD 55 EE 00 00 00 0000 97 01 00 00 00 03 00 01 00 00 01 AA BB

上面指令是把总线上连接的所有控制器的设备地址都修改为 00　01

示例 3：DD 55 EE 00 01 00 0000 97 01 00 00 00 03 00 01 00 00 03 AA BB

上面指令是把总线上原来组地址是 00　01 的控制器的设备地址修改为 00　03

示例 4：DD 55 EE 00 02 00 0300 97 01 00 00 00 03 00 01 00 00 01 AA BB

上面指令是把总线上原来组地址是 00　02，设备地址是 00　03 的控制器的设备地址修改为 00　01

指令的回复：

像上面的示例 2 和示例 3，修改完成后收不到回复，因为是针对所有控制器或者一组控制器进行修改。上面的示例 1 和示例 4，如果有匹配的控制器，修改完后后会收到回复，以字符串形式，示例如下：

0002\r\n

表示设备地址修改完成，新的设备地址是 2。

提示：设备地址修改完成后是立即生效。

2.5.14、显示颜色数据（99）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	00	00	99	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，白色的部分是可能要修改的。

组地址，设备地址，端口号前面已经介绍过，按照需要进行设置即可。

灯带类型支持以下几种取值，按照实际连接的灯带进行选择。不同规格的产品支持的灯带类型有一些差异，下面表格中绿色块代表支持，红色块代表不支持。

代码（HEX）	灯珠型号	固件版本	排针接口款 端子接线款 双端口带壳款	集成USB转串口款	485 串口 6 端口款
01	WS2812B		绿色表示支持		
02	WS2811				
03	WS2813B				
04	SK6812_RGBW				
05	SK6812_RGB				
06	WS2818B				
07	SM16703P				
08	WS2815				
09	SK9822			红色不支持	
0A	DMX512_RGB				
0B	DMX512_RGBW				
0C	GS8208				
0D	UCS1903				
0E	MT1815				
0F	TM1913				
10	TM1914A				
11	SK6812_RGBW_3535				
12	SM16704P_RGBW	>=V1.8			
13	WS2814A_RGBW	>=V1.8			
14	WS2812E_RGB	>=V2.0			
15	TM1934_RGB	>=V2.0			
16	TM1903_RGB	>=V2.0			
17	TM1814_RGBW	>=V2.0			
18	WS2805_RGBCW	>=V2.1			

19	DMX512_C1	>=V2.1			
1A	DMX512_RGBCW	>=V2.1			

(备注：2025 年以后购买的产品固件版本均>=V2.1)

扩展次数的作用如下（在资料中上位机软件使用介绍视频中有更直观的说明）：

当**颜色数据部分**有 3 个字节（即 1 个 LED 的颜色数据）时，如果扩展次数设置的是 00 01，将这条指令发送到灯带上，则只在灯带的第一个 LED 上显示相应的颜色。

当**颜色数据部分**有 3 个字节（即 1 个 LED 的颜色数据）时，如果扩展次数设置的是 00 02，将这条指令发送到灯带上，则在灯带的前两个 LED 上显示相同的颜色。

当**颜色数据部分**有 6 个字节（即 2 个 LED 的颜色数据）时，如果扩展次数设置的是 00 01，将这条指令发送到灯带上，则在灯带的前两个 LED 上显示指令中设置的颜色。

当**颜色数据部分**有 6 个字节（即 2 个 LED 的颜色数据）时，如果扩展次数设置的是 00 02，将这条指令发送到灯带上，则在灯带的前四个 LED 上显示指令中设置的颜色。如果第一个 LED 的颜色是绿色，第二个 LED 的颜色是红色，则第三个 LED 的颜色和第一个 LED 相同，第四个 LED 的颜色和第二个 LED 的颜色相同。

以此类推。

颜色数据部分是实际要发送得到灯带上的数据，对应 RGB 三通道的 LED，每个 LED 需要 3 个字节的数据，分别对应 R、G，B 三个颜色通道的数值的大小。每个字节的取值范围是 00 ~FF，00 最暗，FF 最亮。

比如颜色数据如果是 FF 00 00，将这组颜色显示在 LED 上就是显示最亮的红色。

如果 LED 是 4 通道的 RGBW 四通道的 LED(W 对应白光通道)，则每一个 LED 需要 4 个字节的数据。像 SK6812 的灯带有 3 通道的，也有四通道的。

数据长度部分占 2 个字节，它表示的是**颜色数据**部分总共的字节数。当颜色数据部分有 3 个字节时，数据长度部分的值是 00 03，当颜色数据部分是 6 个字节时，数据长度部分的值是 00 06。因为灯带通常是 3 通道或 4 通道的，所以要求颜色数据部分的总字节数是 3 或者 4 的倍数。这样数据长度部分的值也要对应是 3 或者 4 的倍数。

如果指令中数据长度部分的值和颜色数据部分的字节数不相符，或者不是 4 或者 4 的倍数，则指令无效。

指令的回复：

如果指令中的组地址不是 00 00，设备地址也不是 00 00 ，如果指令正确，控制器接收完指令后

会回复：

RecvEnd\r\n

控制器将颜色数据发送到灯带后会回复：

DisplayEnd\r\n

收到以上两条字符串表示指令被接收并发送到了灯带上。

如果指令中的组地址是 00　00，控制器接收完指令后，将颜色数据发送到灯带上，但是不会回复。

2.5.15、设置上电显示的颜色（9B）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	00	00	9B	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，白色的部分是可能要修改的。

此指令格式和 2.5.10 **显示颜色数据（99）**指令格式完全相同，唯一区别是功能码为 9B。

此指令的作用是设置控制器上电后立即显示的颜色数据。当主机给控制器发送此指令时，指令不会立即在灯带上显示，而是将数据保存。在下一次上电时显示此指令设置的颜色。

指令的回复：

如果指令中的组地址不是 00　00，设备地址也不是 00　00 ，如果指令正确，控制器接收完指令后会回复：

RecvEnd\r\n

指令保存成功后回复：

SaveInsEnd\r\n

指令保存失败的话回复：

SaveInsErr\r\n

2.5.16、关闭上电显示功能（9C）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	00	00	9C	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，白色的部分是可能要修改的。

此指令格式和 2.5.10 **显示颜色数据（99）**指令格式完全相同，唯一区别是功能码为 9C。

此指令的作用是关闭控制器上电后立即显示的颜色数据的功能。当主机给控制器发送此指令时，指令不会立即在灯带上显示。

备注*）既然是关闭上电显示颜色数据功能，那么指令的颜色数据部分的内容并不重要。只需要保证指令格式正确即可。

指令的回复：

如果指令中的组地址不是 00　00，设备地址也不是 00　00 ，如果指令正确，控制器接收完指令后会回复：

RecvEnd\r\n

指令保存成功后回复：

SaveInsEnd\r\n

指令保存失败的话回复：

SaveInsErr\r\n

2.5.17、按 IC 索引修改显示数据（98）（V2.0 固件版本以上才支持）

应用场景：

灯带灯数比较多，而且每次只是修改一个灯或一段灯的颜色状态，可以使用此指令。

如果灯数不多，那么使用**（99）指令**把所有灯颜色数据都列在指令里，每次修改哪些灯就直接修改对应的颜色数据，然后发送指令就可以。不需要使用**（98）指令**。

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	起始 IC 索引		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	00	01	98	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，白色的部分是可能要修改的。

对所有其他指令，红色部分都是保留的，只有此指令红色部分表示要修改的 LED 的 IC 的起始索引，对于 RGB 三通道的灯带，这个值的取值范围是 1~1024。对于 RGBW 四通道的灯带，这个值的范围是 1~768。

组地址，设备地址，端口号前面已经介绍过，按照需要进行设置即可。如果所购买的控制器是单端口控制器，或者购买的是双端口控制器但只使用端口 1，则在指令中将端口号写 01，而不要写 00，这样可以提高程序运行效率。

灯带类型参考前面**（99）指令**的介绍。

扩展次数的作用如下：

当**颜色数据部分**有 3 个字节（即 1 个 LED 的颜色数据）时，如果扩展次数设置的是 00　01，将这条指令发送到灯带上，则只把灯带上起始 IC 索引指定的 LED 修改成颜色数据部分设定的颜色。

当**颜色数据部分**有 3 个字节（即 1 个 LED 的颜色数据）时，如果扩展次数设置的是 00　02，将这条指令发送到灯带上，则把灯带上起始 IC 索引指定的连续 2 个 LED 修改成颜色数据部分设定的颜色。

当**颜色数据部分**有 6 个字节（即 2 个 LED 的颜色数据）时，如果扩展次数设置的是 00　01，将这条指令发送到灯带上，则把灯带上起始 IC 索引指定的连续 2 个 LED 修改成颜色数据部分设定的颜色。

当**颜色数据部分**有 6 个字节（即 2 个 LED 的颜色数据）时，如果扩展次数设置的是 00　02，将这条指令发送到灯带上，则把灯带上起始 IC 索引指定的连续 4 个 LED 修改成颜色数据部分设定的颜色。如果第一个 LED 的颜色是绿色，第二个 LED 的颜色是红色，则第三个 LED 的颜色和第一个 LED 相同，第四个 LED 的颜色和第二个 LED 的颜色相同。

以此类推。

颜色数据参考前面**（99）指令**的介绍。

数据长度部分参考前面**（99）指令**的介绍。

指令的回复：

如果指令中的组地址不是 00　00，设备地址也不是 00　00 ，如果指令正确，控制器接收完指令后会回复：

RecvEnd\r\n

控制器将颜色数据发送到灯带后会回复：

DisplayEnd\r\n

收到以上两条字符串表示指令被接收并发送到了灯带上。

如果指令中的组地址是 00 00，控制器接收完指令后，将颜色数据发送到灯带上，但是不会回复。

注意事项：

注意 1：如果所购买的控制器是单端口控制器，或者购买的是双端口控制器但只使用端口 1，则在指令中将端口号写 01，而不要写 00，这样可以提高程序运行效率。

注意 2：如果是要通过扩展次数把所有灯设定成统一颜色，然后再用（98）指令修改单灯颜色，则要使用（98）指令把所有灯做成统一颜色，不能使用（99）指令。

注意 3：使用（98）指令时，主机也要完整保留全部 LED 的颜色数据，不能依赖控制器保存 LED 的颜色数据。因为一旦控制器断电，数据将丢失。另外如果主机发送指令后回复错误代码 xERR 或回复 timeout，那么控制器中缓存的 LED 数据可能被误修改，最好利用主机保存的数据发送一条完整的颜色数据显示指令还原控制器中的 LED 数据。

注意 4：主机在使用(98)指令对控制器的连接的灯带进行颜色操作过程中，如果中间穿插使用了(9B，9D）指令，则显存里的数据会被修改，若要恢复之前的数据，需要主机利用自身保存的数据用（98，99）指令重新设置灯带的颜色数据。

注意 5：如果一定要使用（98）指令，建议主机程序一直使用（98）指令实现需要的显示功能，不要穿插交替使用（99）指令和（98）指令。

注意 6：TM1814 灯带不支持（98）指令。

2.5.18、读取上电显示的数据（9D）

指令如下：

指令头			组地址		设备地址		端口号	功能码	灯带类型	保留部分		数据长度		扩展次数		颜色数据			指令尾	
DD	55	EE	00	00	00	00	00	9D	01	00	00	00	03	00	01	00	00	00	AA	BB

黄色和蓝色的部分的内容都是固定的，白色的部分是可能要修改的。

此指令格式功能码为 9D。

此指令的作用是读取给控制器设置的上电显示指令。当主机给控制器发送此指令时，控制器通过串口返回上一次通过（9B）指令给控制器设置的上电显示颜色数据。

指令的回复：

如果指令中的组地址不是 00 00，设备地址也不是 00 00 ，如果指令正确，控制器接收完指令后会回复消息，返回的消息分以下几种情况：

a,如果之前给控制器设置过上电显示颜色数据，则返回曾经设置的颜色数据，返回的内容和之前发送的上电显示颜色数据指令完全相同，包含指令头，组地址，等等信息。注意返回的数据以十六进制形式返回。

b,如果之前没有给控制器设置过上电显示颜色数据，或者关闭了上电显示颜色数据，则返回：

null\r\n

指令示例：

示例 1：DD 55 EE 00 00 00 01 00 9D 01 00 00 00 03 00 01 00 00 00 AA BB

上面指令是读取设备地址是 00　01 的控制器的上电显示颜色数据

2.6、灯带类型

名称	灯带类型
字节数	1
索引	9
示例	01

灯带类型占 1 个字节，位于指令中的第 9 字节。

取值范围：0x01~0x1A，这是一个 16 进制数。不同的取值代表不同的灯带。具体如下：

代码（HEX）	**灯珠型号**	**固件版本**	**排针接口款 端子接线款 双端口带壳款**	**集成 USB 转串口款**	**485 串口 6 端口款**
01	WS2812B		绿色表示支持		
02	WS2811				
03	WS2813B				
04	SK6812_RGBW				
05	SK6812_RGB				
06	WS2818B				
07	SM16703P				
08	WS2815				
09	SK9822			红色不支持	
0A	DMX512_RGB				
0B	DMX512_RGBW				
0C	GS8208				
0D	UCS1903				
0E	MT1815				
0F	TM1913				
10	TM1914A				
11	SK6812_RGBW_3535				
12	SM16704P_RGBW	>=V1.8			
13	WS2814A_RGBW	>=V1.8			

14	WS2812E_RGB	>=V2.0			
15	TM1934_RGB	>=V2.0			
16	TM1903_RGB	>=V2.0			
17	TM1814_RGBW	>=V2.0			
18	WS2805_RGBCW	>=V2.1			
19	DMX512_C1	>=V2.1			
1A	DMX512_RGBCW	>=V2.1			

（2025 年之后购买的产品的固件版本均>=V2.1）

注意 1：TM1814 灯带不支持（98）指令。

注意 2：TM1934 灯带，至少每 400ms 发一次指令，否则灯带自己灭灯。

注意 3：TM1814 灯带自带 7 色变化效果，如果不频繁发指令给灯带，灯带就自己运行 7 色变换效果。

注意 4：GS8208 灯带上电自亮，如果不频繁发指令则自动显示默认常亮状态。

注意 5：如果驱动的是 DMX512 系列灯带，则显示颜色指令中控制的灯带点数必须大于等于实际灯带的点数。如果小于实际灯带点数，则末尾的未受控的灯珠颜色会随机变化。

注意 6：不同规格的产品支持的灯带类型有差异，祥见表格中，绿色块代表支持，红色块代表不支持。所以如果实际应用时需要显示静态单色效果，以上灯带非必要不建议使用。

2.7、数据长度

名称	数据长度	
字节数	2	
索引	12	13
示例	00	03

数据长度占 2 个字节，位于指令中的第 12、13 字节。

取值范围：3~3072，这是 10 进制数，使用时要拆分成两个字节，高有效字节在前。比如数据长度是 15，用两个字节 16 进制数表示就是 00　0F。

要求 1：数据长度值必须与指令中颜色数据部分的字节数相同。

要求 2：数据长度值是 3 或者 4 的倍数。

要求 3：如果灯带类型是 5 通道的灯带，比如 WS2805_RGBCW 或者 DMX512_RGBCW。则数据长度值是必须是 5 的倍数。

要求 4：如果驱动的是 DMX512 系列灯带，则显示颜色指令中控制的灯带点数必须大于等于实际灯带的点数。如果小于实际灯带点数，则末尾的未受控的灯珠颜色会随机变化。

2.8、扩展次数

名称	扩展次数	
字节数	2	
索引	14	15
示例	00	01

扩展次数占 2 个字节，位于指令中的第 14、15 字节。扩展次数的作用是把颜色数据复制若干次，进而能使指令驱动更多的灯。比如指令中包含了 8 个 IC 的颜色数据，当扩展次数等于 1 时，指令可以驱动 8 个 IC。当扩展次数等于 2 时，指令可以驱动 16 个 IC。

取值范围：1~1024，这是 10 进制数，使用时要拆分成两个字节，高有效字节在前。比如扩展次数是 15，用两个字节 16 进制数表示就是 00　0F。

注意：扩展次数 x 数据长度不能大于 3072

2.9、颜色数据

名称	颜色数据		
字节数	3~3072		
索引	16	17	18
示例	FF	00	00

颜色数据占 3~3072 个字节，在扩展次数之后，尾部之前都是颜色数据。

比如连接的是 3 通道的 LED，要在灯带上显示三个最亮的红色，则颜色数据部分是：

FF　00　00　FF　00　00　FF　00　00

如果想显示不太亮的三个红色，则颜色数据可以是：

25　00　00　25　00　00　25　00　00

如果想第一个 LED 亮红色，第二个亮绿色，第三个亮蓝色，则颜色数据可以是：

FF　00　00　00　FF　00　00　00　FF

如果要让灯带灭灯的，将颜色数据的 RGB 通道值都写 00，黑色就是灭灯

00　00　00　00　00　00　00　00　00

2.10、当指令错误时控制器的回复内容

控制器在接收指令的过程中如果发现指令不符合指令的格式要求，会回复相应的错误码，如下：

错误码	含义
HERR\r\n	指令头错误
GERR\r\n	组地址错误（超出最大值 1024）
AERR\r\n	设备地址错误（超出最大值 4096）
PERR\r\n	端口错误（超出最大值 6）
CERR\r\n	功能码错误
IERR\r\n	灯带类型错误
LERR\r\n	数据长度错误
RERR\r\n	扩展次数错误
TERR\r\n	指令尾部错误
DERR\r\n	数据长度与颜色数据字节数不符

如果主机发送给控制器的指令不完整，或者主机发送给控制的指令不是一次性发送，会导致控制器接收指令超时，此时回复：

Timeout\r\n

出现以上任何错误信息回复时请仔细检查指令。

注意：主机给控制器发送指令的时候，要确保一条指令发送完成后再发送下一条指令。如果上一条指令还没有发送完就发送新的指令，指令不会被控制器正确接收，也会导致回复错误信息。

三、颜色显示指令范例

3.1、WS2812B 三通道 RGB 灯带示例

示例 1：组地址 0000、 设备地址 0001、端口 00、包含 1 个 LED 数据，扩展 1 次：

DD 55 EE 00 00 00 01 00 99 01 00 00 00 03 00 01 50 00 00 AA BB

示例 2：组地址 0000、 设备地址 0001、端口 00、包含 2 个 LED 数据，扩展 1 次：

DD 55 EE 00 00 00 01 00 99 01 00 00 00 06 00 01 50 00 00 50 00 00AA BB

示例 3：组地址 0001、 设备地址 0001、端口 00、包含 1 个 LED 数据，扩展 8 次 ：

DD 55 EE 00 01 00 01 00 99 01 00 00 00 03 00 08 50 00 00 AA BB

示例 4：组地址 0001、 设备地址 0002、端口 00、包含 1 个 LED 数据，扩展 8 次：

DD 55 EE 00 01 00 02 00 99 01 00 00 00 03 00 08 50 00 00 AA BB

示例 5：所有组、 所有设备、端口 00、包含 1 个 LED 数据，重复 8 次 ：

DD 55 EE00 00 00 00 00 99 01 00 00 00 03 00 08 50 00 00 AA BB

3.2、SK6812_RGBW 四通道 RGBW 灯带示例

示例 1：组地址 0000、 设备地址 0001、端口 00、显示 1 个 LED 数据，重复 1 次 ：

DD 55 EE 00 00 00 01 00 99 04 00 00 00 04 00 01 50 00 00 00 AA BB

示例 2：组地址 0000、 设备地址 0001、端口 00、显示 2 个 LED 数据，重复 1 次 ：

DD 55 EE 00 00 00 01 00 99 04 00 00 00 08 00 01 50 00 00 00 50 00 00 00 AA BB

示例 3：组地址 0001、 设备地址 0001、端口 01、显示 1 个 LED 数据，重复 1 次 ：

DD 55 EE 00 01 00 01 01 99 04 00 00 00 04 00 01 50 00 00 00 AA BB

示例 4：组地址 0001、 设备地址 0001、端口 02、显示 1 个 LED 数据，重复 1 次 ：

DD 55 EE 00 01 00 01 02 99 04 00 00 00 04 00 01 50 00 00 00 AA BB

以上指令中颜色数据都很少，实际上颜色数据可以很多，比如下面这条：

示例 1：组地址 0000，设备地址 0001、端口 00，显示 100 个 LED 数据-绿色，重复 1 次，颜色数据 50 是中等亮度，最亮是 FF，不亮是 00：

DD 55 EE 00 00 00 01 00 99 01 00 00 01 2C 00 01 00 50 00 00 50 00 00 50 00 00 50 00 00 50 00 00

50 00 00 50 00 00 50 00 00

```
//定义一个组地址变量
Uint16_t        groupAddress    =    0;
LedArr[3] = (Uint8_t) (groupAddress>> 8);
LedArr[4] = (Uint8_t) groupAddress;

//设备地址赋值
//定义一个设备地址变量
Uint16_t        deviceAddress    =    1;
LedArr[5] = (Uint8_t) (deviceAddress>> 8);
LedArr[6] = (Uint8_t) deviceAddress;


//端口号赋值
//定义一个设备地址变量
Uint8_t    port    =    0;
LedArr[7] = port;

//功能码赋值
LedArr[8] = 0x99;

//灯带类型赋值
LedArr[9] = 0x01;

//保留的两个字节赋值
LedArr[10] = 0x00;
LedArr[11] = 0x00;

//数据长度赋值
//定义一个数据长度变量
Uint16_t        dataLength    =    30*3;
LedArr[12] = (Uint8_t) (dataLength>> 8);
LedArr[13] = (Uint8_t) dataLength;

//扩展次数赋值
//定义一个扩展次数变量
Uint16_t        repeat    =    1;
LedArr[14] = (Uint8_t) (repeat>> 8);
LedArr[15] = (Uint8_t) repeat;

//尾部字节赋值
LedArr[108 - 2] = 0xAA;
LedArr[108 - 1] = 0xBB;

//下面是颜色数据部分，可以利用循环结构进行赋值
//比如所有 LED 都显示红色
for(uint16_t    m = 0; m < 30; m++)
{
    for(uint16_t    n = 0; n < 3; n++)
    {
```

```
            Switch(n)
            {
                //R 通道
                Case 0:
                {
                    //16+是因为，颜色数据是从数组的索引的 16 号开始的
                    //m*3 是因为每个 IC 需要 3 个字节数据
                    LedArr[16 + (m * 3) + n] = 0xFF;
                    Break;
                }

                //G 通道
                Case 1:
                {
                    //16+是因为，颜色数据是从数组的索引的 16 号开始的
                    //m*3 是因为每个 IC 需要 3 个字节数据
                    LedArr[16 + (m * 3) + n] = 0x00;
                    Break;
                }
                //B 通道
                Case 2:
                {
                    //16+是因为，颜色数据是从数组的索引的 16 号开始的
                    //m*3 是因为每个 IC 需要 3 个字节数据
                    LedArr[16 + (m * 3) + n] = 0x00;
                    Break;
                }
            }
        }
}
```

这样将整个数组都赋值完成后，发数组通过串口发送到控制器，就可以在灯带上显示相应的颜色。

如果想让 30 个 IC 显示由暗到亮的红色，可以参考下面这样赋值，只需要改下面红色标记的地方。

```
for(uint16_t    m = 0; m < 30; m++)
{
    for(uint16_t    n = 0; n < 3; n++)
    {
        Switch(n)
        {
            //R 通道
            Case 0:
            {
                //16+是因为，颜色数据是从数组的索引的 16 号开始的
                //m*3 是因为每个 IC 需要 3 个字节数据
```

```
                LedArr[16 + (m * 3) + n] = m*3;
                Break;
            }

            //G 通道
            Case 1:
            {
                //16+是因为，颜色数据是从数组的索引的 16 号开始的
                //m*3 是因为每个 IC 需要 3 个字节数据
                LedArr[16 + (m * 3) + n] = 0x00;
                Break;
            }
            //B 通道
            Case 2:
            {
                //16+是因为，颜色数据是从数组的索引的 16 号开始的
                //m*3 是因为每个 IC 需要 3 个字节数据
                LedArr[16 + (m * 3) + n] = 0x00;
                Break;
            }
        }
    }
}
```

如果想让 30 个 IC 显示显示一个红色的流水效果，怎么实现呢?

首先如果要实现流水效果，要连续发指令，每改一次数据，发一次指令

可以像下面这样:

```
for(uint16_t    m = 0; m < 30; m++)
{
    //这里 30*3 是因为 30 个 IC，每个 IC 占 3 个字节，一共 30*3 个字节
    //首先把 LED 颜色数据都清零
    for(uint16_t    k = 0; k < 30*3; k++)
    {
        LedArr[16 + k] = 0x00;
    }
    //然后给颜色数据赋值
    for(uint16_t    n = 0; n < 3; n++)
    {
        Switch(n)
        {
            //R 通道
            Case 0:
            {
                //16+是因为，颜色数据是从数组的索引的 16 号开始的
                //m*3 是因为每个 IC 需要 3 个字节数据
```

```
                LedArr[16 + (m * 3) + n] =0xFF;
                Break;
            }

            //G 通道
            Case 1:
            {
                //16+是因为，颜色数据是从数组的索引的 16 号开始的
                //m*3 是因为每个 IC 需要 3 个字节数据
                LedArr[16 + (m * 3) + n] = 0x00;
                Break;
            }
            //B 通道
            Case 2:
            {
                //16+是因为，颜色数据是从数组的索引的 16 号开始的
                //m*3 是因为每个 IC 需要 3 个字节数据
                LedArr[16 + (m * 3) + n] = 0x00;
                Break;
            }
        }
    }
    //通过串口发送指令
    //Serial.write(&LedArr[0],108);
    //延时一段时间
    Delay(100);
}
```

五、其他说明

控制器每次启动时都会发出一串字符，刚出厂的控制器发送的是：

“0000_0001_powerON\r\n”

其中前四位数字代表控制器的组地址，后面四位数字代表控制器的设备地址。

如果连接好控制器后，给控制器上电，没有收到这串字符，那么说明通信线不通或接错。如果收到的是乱码，那么可能主机选择的波特率与控制器的波特率不匹配。

如果控制器固件版本大约 V2.0，则收到前面字符串之后，会紧跟着收到另一串字符：

“S2000_0005_V2.0_F0\r\n”

其中 S2000 代表的是控制器型号，0005 代表的是控制器的串口通信超时时间（详细说明见 8D 和 8E 指令），V2.0 代表控制器固件版本号，F0 代表的是控制器反馈信息设置（详细说明见 8B 和 8C 指令）。

六、联系我们

您也可以通过以下方式与我们取得联系：

旺旺：和我联系QQ：377347768

如果您需要其他相关产品，欢迎访问我们的淘宝旗舰店！

淘宝旗舰店：http://flyelectronic.taobao.com/

天津创联容微科技有限公司

专业的电子技术软硬件开发制造商 ...make electronics easy to use and learn.

访问更多产品

淘宝旗舰店：http://flyelectronic.taobao.com

联系我们

E-mail：377347768@qq.com QQ：377347768